Multiplication

Decimal multiplication

(carry) 1_
```
  37
x 12
----
  74
 370
----
 444
```

- What are the rules?
  - Successively multiply the multiplicand by each digit of the multiplier starting at the right shifting the result left by an extra left position each time keeping the first bit (LSB) as 0.
  - Sum all partial results

Binary multiplication

23	10111	Multiplicand
19	× 10011	Multiplier
	10111	
	10111	
	00000 +	
	00000	
	10111	
437	110110101	Product

- Multiplication of two fixed point binary numbers in signed magnitude representation is done with paper and pencil of successive shift and add operation.
- If the multiplier bit is a 1, the multiplicand is copied down; otherwise zeroes are copied down.

Binary multiplication…

- When multiplication is implemented in a digital computer, it is convenient to change the process slightly.

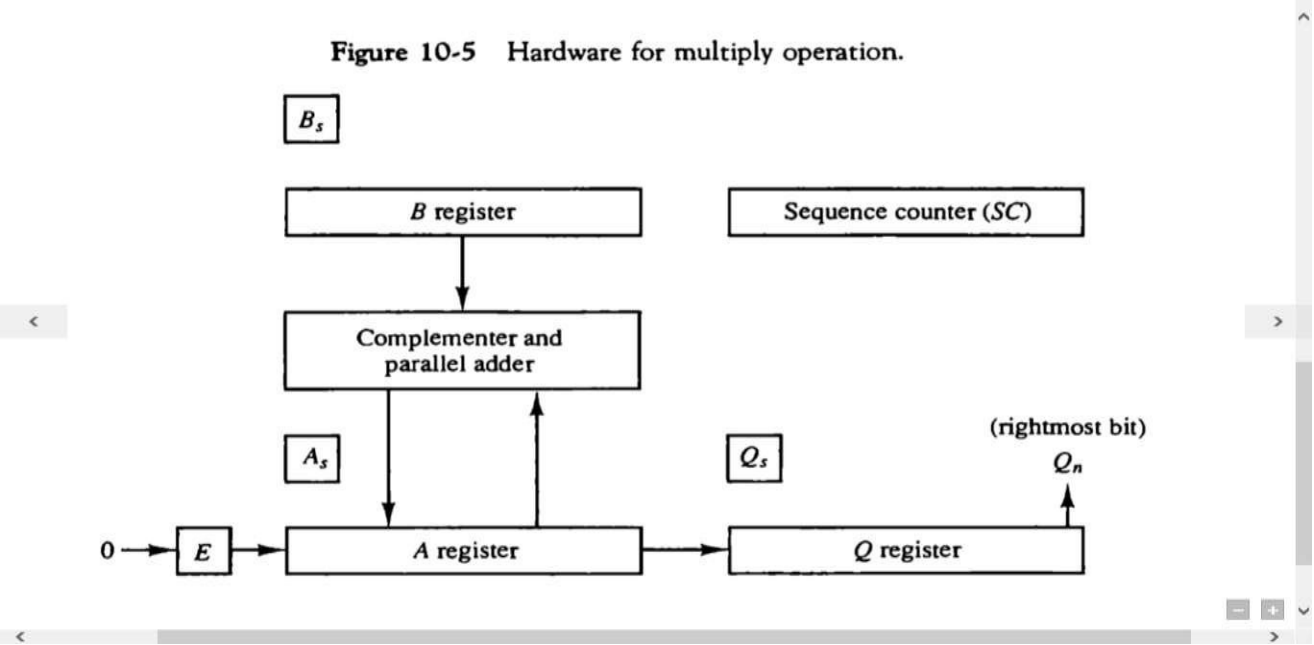
- First instead of providing register to store and add simultaneously as many binary numbers as there are bits in the multiplier, as it is convenient to provide an adder for the summation of only two binary numbers and successively accumulate the partial products in a register.

- Second instead of shifting the multiplicand to the left, the partial product is shifted to the right.

Binary multiplication…

- The hardware for multiplication consists of the equipment shown in following figure plus two more registers.
- These registers are together with registers A and B.
- The multiplier stored in the Q register and its sign in Q_s.
- The sequence counter SC is initially set to a number equal to the number of bits in the multiplier. The counter is decremented by 1 after forming each partial product.
- The sum of A and B forms a partial product which is transferred to the EA register .

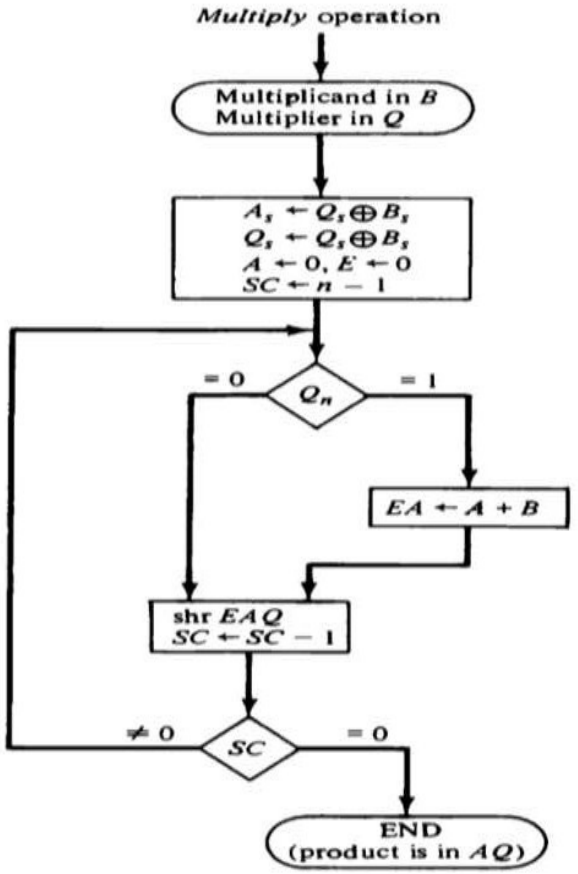
Binary multiplication…



- The shift will be denoted by the statement shr EAQ to designate the right shift depicted.
- The least significant bit of A is shifted into the most significant position of Q.

Binary multiplication…

Figure 10-6 Flowchart for multiply operation.



Binary multiplication…

- When multiplication is implemented in a digital computer, it is convenient to change the process slightly.
- First instead of providing register to store and add simultaneously as many binary numbers as there are bits in the multiplier, it is convenient to provide an adder for the summation of only two binary numbers and successively accumulate the partial products in a register.
- Second instead of shifting the multiplicand to the left, the partial product is shifted to the right.
- The sequence counter SC is initially set to a number equal to the number of bits in the multiplier.
- The counter is decremented by 1 after forming each partial product.
- The sum of A and B forms a partial product which is transferred to the EA register .

Binary multiplication…

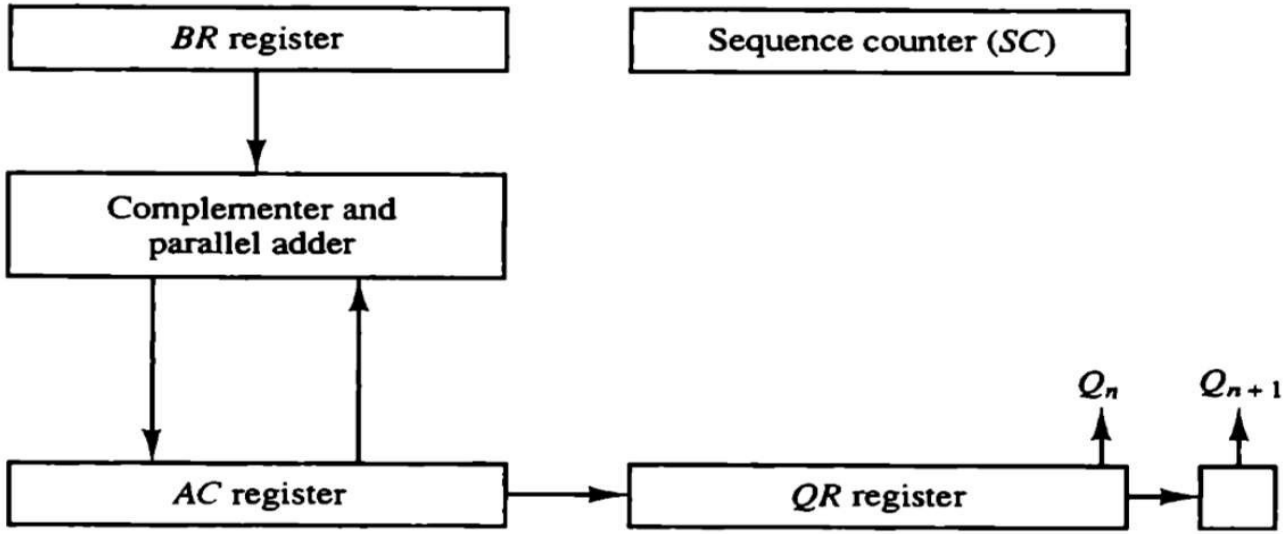
TABLE 10-2 Numerical Example for Binary Multiplier

Multiplicand $B = 10111$	E	A	Q	SC
Multiplier in Q	0	00000	10011	101
$Q_n = 1$; add B		10111		
First partial product	0	10111		
Shift right EAQ	0	01011	11001	100
$Q_n = 1$; add B		10111		
Second partial product	1	00010		
Shift right EAQ	0	10001	01100	011
$Q_n = 0$; shift right EAQ	0	01000	10110	010
$Q_n = 0$; shift right EAQ	0	00100	01011	001
$Q_n = 1$; add B		10111		
Fifth partial product	0	11011		
Shift right EAQ	0	01101	10101	000
Final product in $AQ = 0110110101$				

Booth's Multiplication Algorithm

Booth Algorithm gives a procedure for multiplying binary integers in signed-2's complement representation .

Figure 10-7 Hardware for Booth algorithm.



Booth's Multiplication Algorithm…

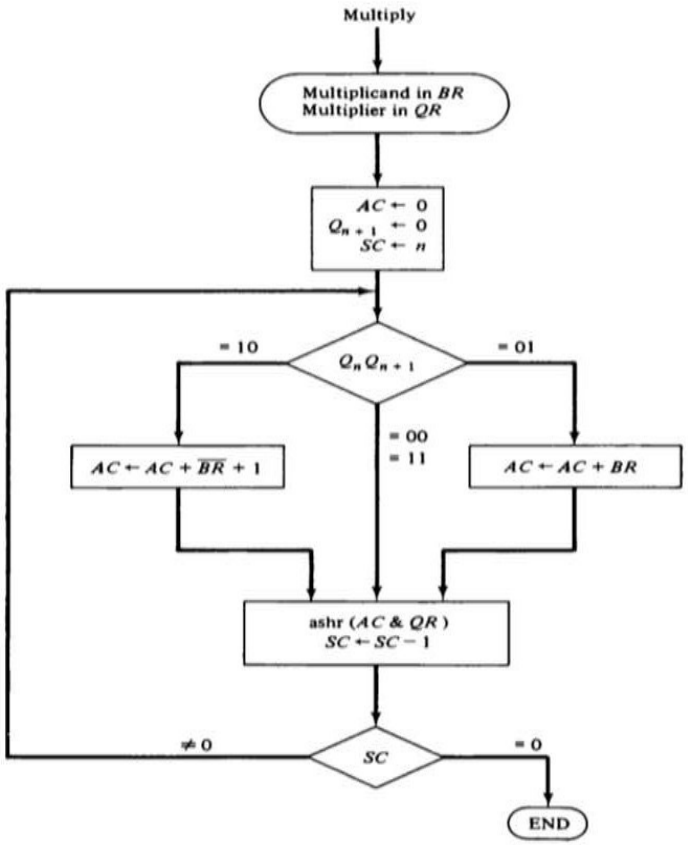


Figure 10-8 Booth algorithm for multiplication of signed-2's complement numbers.

Booth's Multiplication Algorithm...

Initialization:

1. `AC` = 0
2. `Qn+1` = 0
3. `QR` = Multiplier
4. `BR` = Multiplicand
5. `SC` = n (number of bits)

Iteration (repeated until SC = 0):

At each step, the bits `(Qn, Qn+1)` (the least significant bit of QR and the Qn+1 flip-flop) are examined.

1. **If (Qn, Qn+1) = 00:** Perform only an **arithmetic right shift** on the combined `[AC, QR, Qn+1]` register.
2. **If (Qn, Qn+1) = 01: Add** the multiplicand (`BR`) to `AC` (`AC = AC + BR`), then perform an **arithmetic right shift** on the combined `[AC, QR, Qn+1]` register.
3. **If (Qn, Qn+1) = 10: Subtract** the multiplicand (`BR`) from `AC` (`AC = AC - BR`), then perform an **arithmetic right shift** on the combined `[AC, QR, Qn+1]` register.
4. **If (Qn, Qn+1) = 11:** Perform only an **arithmetic right shift** on the combined `[AC, QR, Qn+1]` register.

Decrement:

1. Decrement the sequence counter (`SC = SC - 1`) after each shift operation.

Result:

1. After the loop finishes (when `SC` reaches 0), the final product (a $2n$-bit signed number) is stored in the concatenated `[AC, QR]` registers. The leftmost bit of AC preserves the sign throughout the operation due to the arithmetic shift.

Booth's Multiplication Algorithm…

TABLE 10-3 Example of Multiplication with Booth Algorithm

$Q_n\ Q_{n+1}$	$BR = 10111$ $\overline{BR} + 1 = 01001$	AC	QR	Q_{n+1}	SC
	Initial	00000	10011	0	101
1 0	Subtract BR	01001			
		01001			
	ashr	00100	11001	1	100
1 1	ashr	00010	01100	1	011
0 1	Add BR	10111			
		11001			
	ashr	11100	10110	0	010
0 0	ashr	11110	01011	0	001
1 0	Subtract BR	01001			
		00111			
	ashr	00011	10101	1	000